



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №6

по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушин С.М.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Метод решения	4
1.1 Задание 1. Кластеризация без снижения размерности	6
1.2 Задание 2. Снижение размерности	16
1.3 Задание 3. Влияние снижения размерности	19
Вывод.....	22

ВВЕДЕНИЕ

Цель практической работы №6 — изучить алгоритмы кластеризации и методы снижения размерности, освоить их применение, а также понять, как снижение размерности влияет на работу кластеризационных алгоритмов.

1 МЕТОД РЕШЕНИЯ

Необходимо подобрать датасет, в котором не менее 1000 строк, хотя бы 4 числовых признака и 2 категориальных. Импорт представлен в Листинге 1.

Листинг 1 — Импорт библиотек и датасета

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

df = pd.read_csv('ship.csv')
df.head()
```

Так выглядят первые 5 строк датасета (Рисунок 1).

	Date	Ship_Type	Route_Type	Engine_Type	Maintenance_Status	Speed_Over_Ground_knots	Engine_Power_kW	Distance_Traveled_nm	Draft_meters	Weather_Condition
0	2023-06-04	Container Ship	NaN	Heavy Fuel Oil (HFO)	Critical	12.597558	2062.983982	1030.943616	14.132284	Moderate
1	2023-06-11	Fish Carrier	Short-haul	Steam Turbine	Good	10.387580	1796.057415	1060.486382	14.653083	Rough
2	2023-06-18	Container Ship	Long-haul	Diesel	Fair	20.749747	1648.556685	658.874144	7.199261	Moderate
3	2023-06-25	Bulk Carrier	Transoceanic	Steam Turbine	Fair	21.055102	915.261795	1126.822519	11.789063	Moderate
4	2023-07-02	Fish Carrier	Transoceanic	Diesel	Fair	13.742777	1089.721803	1445.281159	9.727833	Moderate

Рисунок 1 — Первые 5 строк датасета

Сам датасет имеет 18 признаков и 2736 строк.

Посмотрим на наличие пропусков в датасете (Рисунок 2).

```
Date          0
Ship_Type      136
Route_Type     136
Engine_Type    136
Maintenance_Status 136
Speed_Over_Ground_knots 0
Engine_Power_kW 0
Distance_Traveled_nm 0
Draft_meters   0
Weather_Condition 136
Cargo_Weight_tons 0
Operational_Cost_USD 0
Revenue_per_Voyage_USD 0
Turnaround_Time_hours 0
Efficiency_nm_per_kWh 0
Seasonal_Impact_Score 0
Weekly_Voyage_Count 0
Average_Load_Percentage 0
dtype: int64
```

Рисунок 2 — Пропуски в датасете

Замечаем, что сразу в пяти признаках обнаружено по 136 пропусков. Заполним их самым часто встречающимся значением, то есть модой (Листинг 2).

Листинг 2 — Заполнение пропусков

```
df['Route_Type'].fillna(df['Route_Type'].mode()[0], inplace=True)
df['Ship_Type'].fillna(df['Ship_Type'].mode()[0], inplace=True)
df['Engine_Type'].fillna(df['Engine_Type'].mode()[0], inplace=True)
df['Maintenance_Status'].fillna(df['Maintenance_Status'].mode()[0],
inplace=True)
df['Weather_Condition'].fillna(df['Ship_Type'].mode()[0], inplace=True)
```

После того, как заполнили пропуски, переходим к обработке выбросов. Выполним обработку уже известным методом IQR (Листинг 3).

Листинг 3 — Обработка выбросов

```
def remove_outliers_iqr(df, columns):
    df_clean = df.copy()
    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df_clean = df_clean[(df_clean[col] >= lower_bound) & (df_clean[col] <=
upper_bound)]
    return df_clean

numerical_cols = [
    'Speed_Over_Ground_knots', 'Engine_Power_kW', 'Distance_Traveled_nm',
    'Draft_meters', 'Cargo_Weight_tons', 'Operational_Cost_USD',
    'Revenue_per_Voyage_USD', 'Turnaround_Time_hours', 'Efficiency_nm_per_kWh',
    'Seasonal_Impact_Score', 'Weekly_Voyage_Count', 'Average_Load_Percentage'
]
df_no_outliers = remove_outliers_iqr(df, numerical_cols)
print(f"Было: {len(df)}, стало: {len(df_no_outliers)}")
df = df_no_outliers.reset_index(drop=True)
```

Выбросы успешно обработаны, переходим к кодировке категориальных признаков (Листинг 4).

Листинг 4 — Кодировка категориальных признаков

```
categorical_cols = ['Ship_Type', 'Route_Type', 'Engine_Type',
'Maintenance_Status', 'Weather_Condition']
le_dict = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    le_dict[col] = le
```

И выполним масштабирование числовых признаков. Код программы представлен в Листинге 5.

Листинг 5 — Масштабирование числовых признаков

```
df_cluster = df.drop(columns=['Date'])
X = df_cluster.copy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
```

После того, как была проведена первичная предобработка данных, переходим к кластеризации.

1.1 Задание 1. Кластеризация без снижения размерности

Обнаружим оптимальное количество кластеров при помощи двух методов: метода локтя и коэффициента силуэта.

Сперва сделаем метод локтей, результатом будет график зависимости суммы квадратов внутрикластерных расстояний от числа кластеров (Листинг 6).

Листинг 6 — Метод локтей

```
wcss = []
K_range = range(2, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
plt.plot(K_range, wcss, marker='o')
plt.title('Метод локтя')
plt.xlabel('Количество кластеров k')
plt.ylabel('WCSS')
plt.grid()
plt.show()
```

График изображен на Рисунке 3.

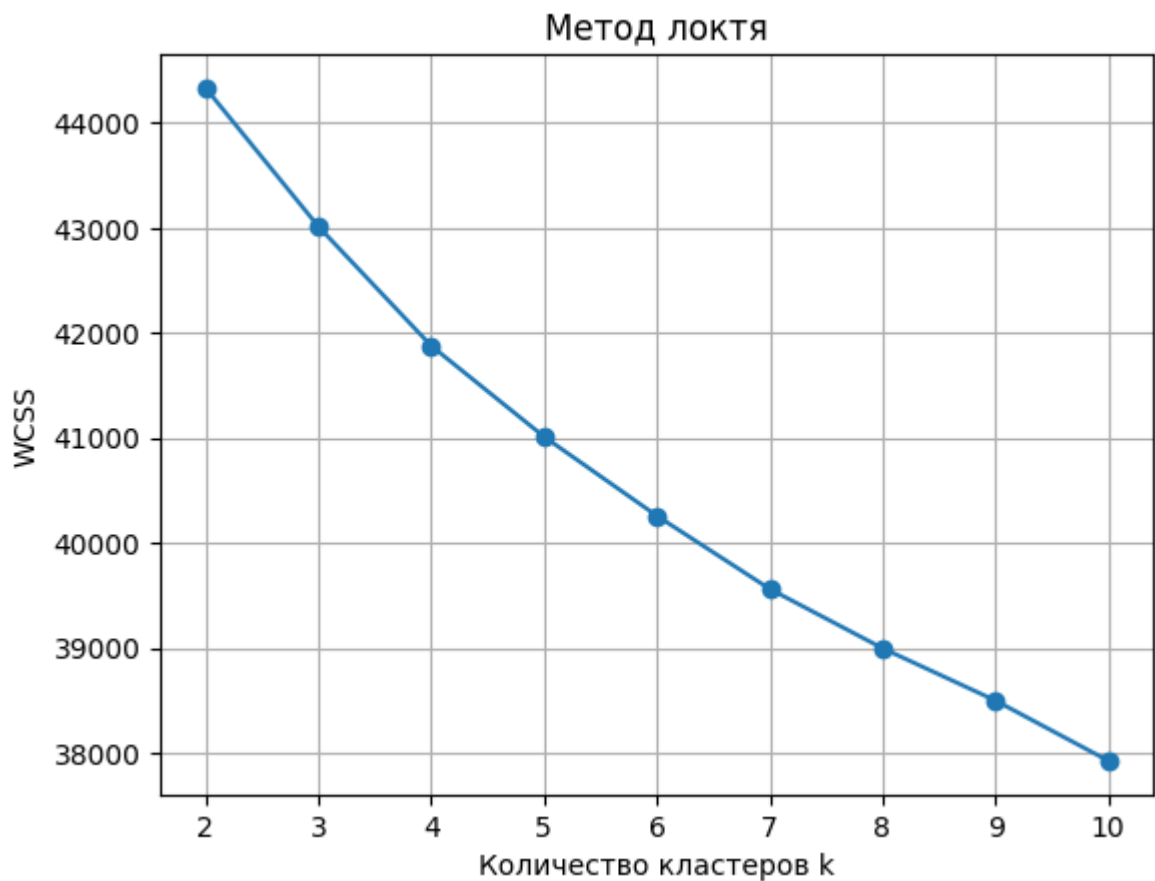


Рисунок 3 — Метод локтя

Заметим, что с увеличением числа кластеров плавно уменьшается внутрикластерное расстояние. Это говорит о том, что нет ярковыраженных кластеров в выборке.

Перейдем к коэффициенту силуэта (Листинг 7).

Листинг 7 — Коэффициент силуэта

```
sil_scores = []
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    sil_avg = silhouette_score(X_scaled, labels)
    sil_scores.append(sil_avg)
plt.plot(K_range, sil_scores, marker='o', color='orange')
plt.title('Коэффициент силуэта')
plt.xlabel('Количество кластеров k')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.show()
best_k = K_range[np.argmax(sil_scores)]
print(f"Лучшее k по силуэту: {best_k}")
```

Посмотрим на график (Рисунок 4).



Рисунок 4 — Коэффициент силуэта

График показывает, что при количестве кластеров, равном двум, коэффициент силуэта достигает своего максимального значения, однако это значение ближе к нулю, чем к единице, что так же говорит о том, что структура представляет собой одно сплошное облако точек с некоторой внутренней вариативностью. Тем не менее, оптимальное количество кластеров равно двум. Выполним кластеризацию по лучшему k и посмотрим на посчитанные метрики (Листинг 8).

Листинг 8 — Метрики качества

```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
start_time = time.time()
labels_kmeans = kmeans.fit_predict(X_scaled)
time_kmeans = time.time() - start_time
sil_kmeans = silhouette_score(X_scaled, labels_kmeans)
db_kmeans = davies_bouldin_score(X_scaled, labels_kmeans)
ch_kmeans = calinski_harabasz_score(X_scaled, labels_kmeans)
print(f"KMeans метрики (k={best_k}):")
print(f"Silhouette: {sil_kmeans:.3f}")
print(f"Davies-Bouldin: {db_kmeans:.3f}")
print(f"Calinski-Harabasz: {ch_kmeans:.1f}")
print(f"Время обучения: {time_kmeans:.2f} сек")
```


Обратим внимание на результаты:

1. Davies-Bouldin Index: 4.472 — высокое значение (чем меньше — тем лучше). Это подтверждает плохое качество кластеризации: кластеры не компактны и/или плохо разделены.
2. Calinski-Harabasz Index: 134.4 — относительно низкое значение (чем больше — тем лучше). Оно указывает на слабую межкластерную дисперсию по сравнению с внутрикластерной.

Выполним визуализацию кластеров (Листинг 9).

Листинг 9 — График

```
pca_viz = PCA(n_components=2)
X_pca_viz = pca_viz.fit_transform(X_scaled)
plt.scatter(X_pca_viz[:, 0], X_pca_viz[:, 1], c=labels_kmeans, cmap='viridis',
            alpha=0.7)
plt.title('K-means кластеры (PCA для визуализации)')
plt.colorbar(label='Кластер')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Визуализация представлена на Рисунке 5.

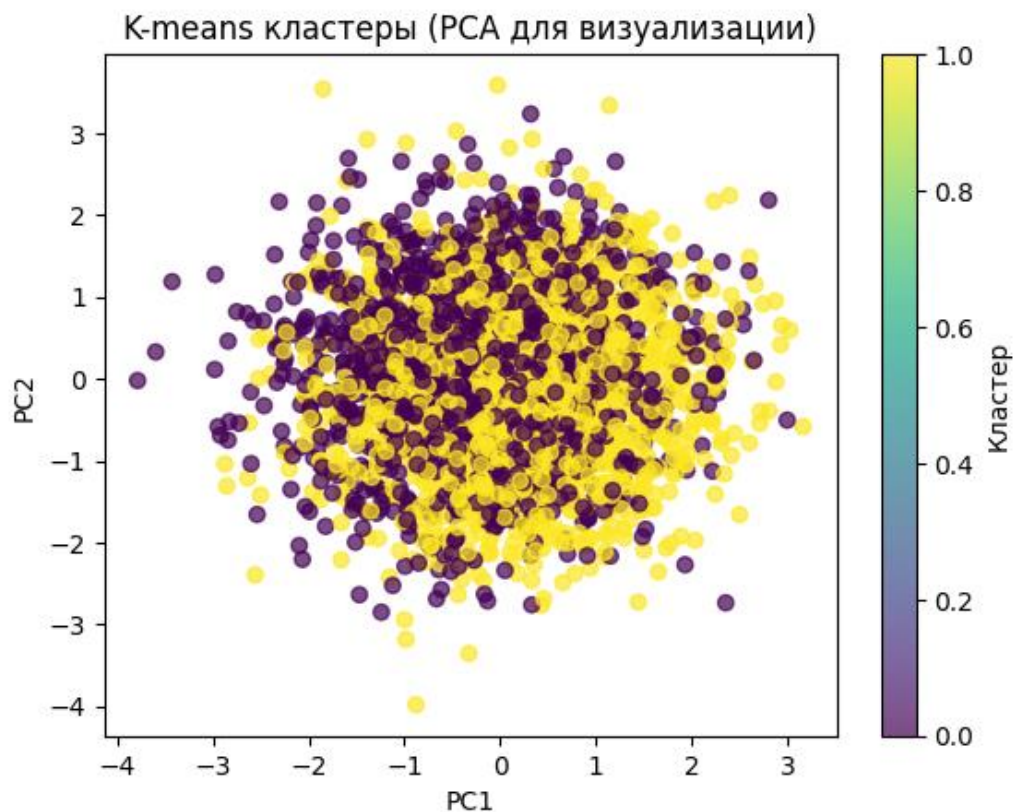


Рисунок 5 — Визуализация

Точки двух кластеров сильно пересекаются и не образуют отдельных скоплений. Нет чётких границ между группами. Это визуально подтверждает низкие значения метрик качества.

Переходим к иерархической кластеризации. В нем мы сравним два метода связывания: `wade` и `average`. Первый требует евклидово расстояние и работает только с полными данными, второй же метод более гибок. Но сперва построим дендрограмму (Листинг 10).

Листинг 10 — Дендрограмма

```
linkage_full = linkage(X_scaled, method='ward')
plt.figure(figsize=(14, 8))
dendrogram(linkage_full, truncate_mode='level', p=5, show_contracted=True)
plt.title('Дендрограмма')
plt.xlabel('Кластеры / Объекты')
plt.ylabel('Расстояние (несходство)')
plt.grid(True, alpha=0.3)
plt.show()
```

Дендрограмма изображена на Рисунке 6.

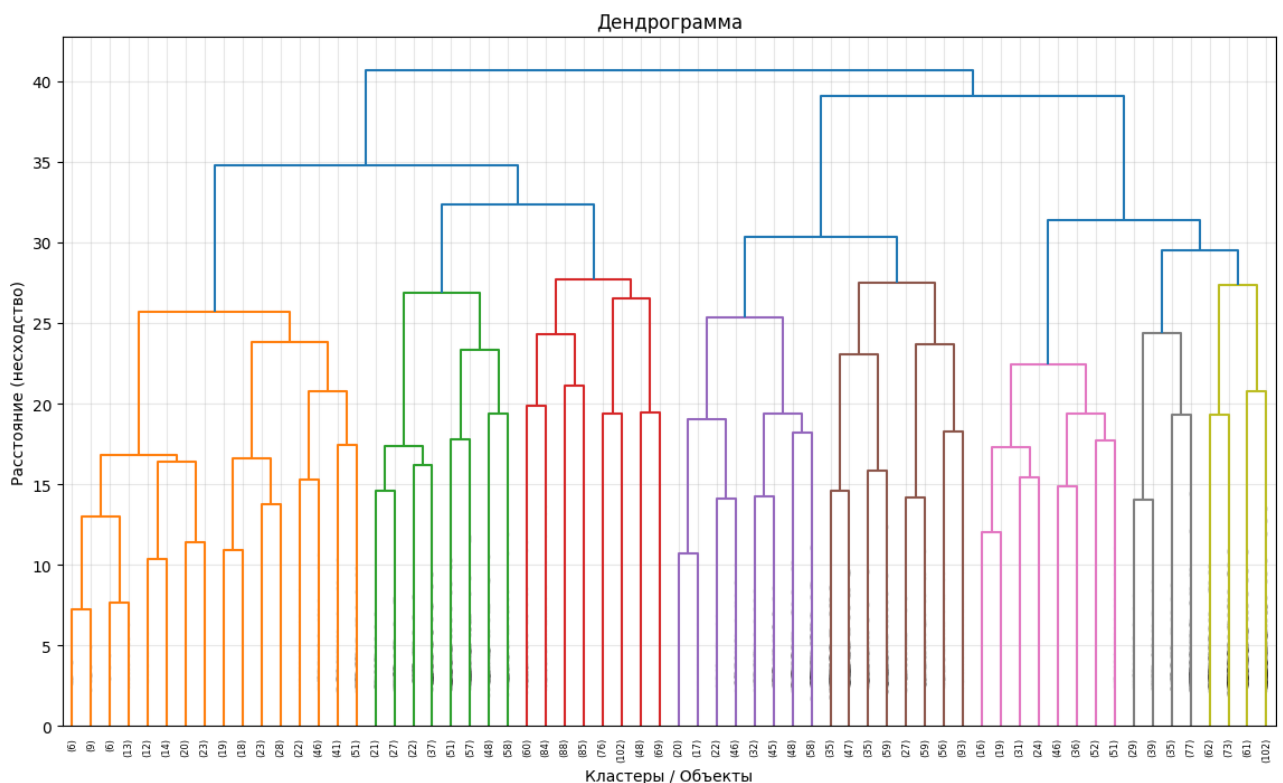


Рисунок 6 — Дендрограмма

Перейдем к реализации метода иерархической кластеризации (Листинг 11).

Листинг 11 — Иерархическая кластеризация

```
from sklearn.cluster import AgglomerativeClustering

def evaluate_hierarchical(X, k_range, linkage='ward'):
    sil_scores = []
    db_scores = []
    ch_scores = []
    times = []
    for k in k_range:
        start = time.time()
        agg = AgglomerativeClustering(n_clusters=k, linkage=linkage)
        labels = agg.fit_predict(X)
        end = time.time()
        sil = silhouette_score(X, labels)
        db = davies_bouldin_score(X, labels)
        ch = calinski_harabasz_score(X, labels)
        sil_scores.append(sil)
        db_scores.append(db)
        ch_scores.append(ch)
        times.append(end - start)
    return sil_scores, db_scores, ch_scores, times

k_range = range(2, 11)
sil_ward, db_ward, ch_ward, time_ward = evaluate_hierarchical(X_scaled,
k_range, linkage='ward')
sil_avg, db_avg, ch_avg, time_avg = evaluate_hierarchical(X_scaled, k_range,
linkage='average')
best_k_ward = k_range[np.argmax(sil_ward)]
best_sil_ward = max(sil_ward)
best_k_avg = k_range[np.argmax(sil_avg)]
best_sil_avg = max(sil_avg)
print(f"Ward: лучшее k = {best_k_ward}, Silhouette = {best_sil_ward:.3f}")
print(f"Average: лучшее k = {best_k_avg}, Silhouette = {best_sil_avg:.3f}")
```

Посмотрим на результат:

- ward: лучшее k = 2, Silhouette = 0.018;
- average: лучшее k = 2, Silhouette = 0.095.

Поскольку у метода average коэффициент силуэта выше, значит этот метод связывания лучше справляется. Посчитаем для него метрики (Листинг 12).

Листинг 12 — Метрики иерархической кластеризации (average)

```
best_linkage = 'average'
best_k_hier = best_k_ward
agg_final = AgglomerativeClustering(n_clusters=best_k_hier,
linkage=best_linkage)
start = time.time()
labels_hier = agg_final.fit_predict(X_scaled)
time_hier = time.time() - start
sil_hier = silhouette_score(X_scaled, labels_hier)
db_hier = davies_bouldin_score(X_scaled, labels_hier)
ch_hier = calinski_harabasz_score(X_scaled, labels_hier)
print(f"Итог иерархической кластеризации ({best_linkage}, k={best_k_hier}):")
print(f"Silhouette: {sil_hier:.3f}")
print(f"Davies-Bouldin: {db_hier:.3f}")
print(f"Calinski-Harabasz: {ch_hier:.1f}")
```

```
print(f"Время: {time_hier:.2f} сек")
```

Получились следующие результаты:

1. Davies-Bouldin: 1.640 — это значительно лучше, чем у K-means (4.472) и ward (4.45). Чем ниже значение, тем лучше — значит, кластеры компактнее и лучше разделены.
2. Calinski-Harabasz: 3.1 — значение очень низкое, что говорит о слабой межкластерной дисперсии.

Посмотрим на визуализацию (Рисунок 7).

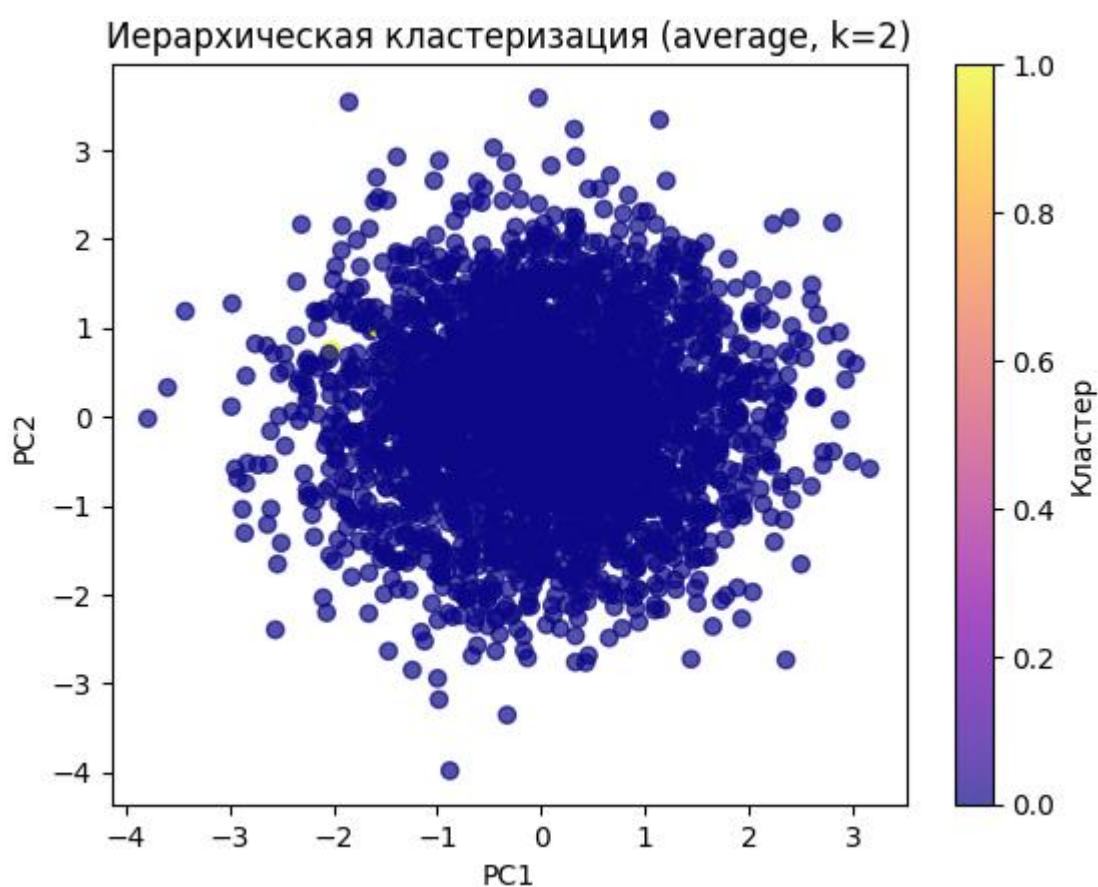


Рисунок 7 — Визуализация иерархической кластеризации

Визуально кластеры выглядят как одно большое скопление точек с едва заметным разделением. Большинство точек окрашено в один цвет (тёмно-синий), что соответствует одному кластеру, а небольшая часть — в светло-жёлтый (второй кластер). Это подтверждает низкий коэффициент силуэта — границы

между кластерами очень размыты, и большинство объектов находятся в "переходной зоне" между группами.

Перейдем к методу DBSCAN. В отличие от K-means и иерархической кластеризации, DBSCAN не требует задавать число кластеров заранее и умеет находить кластеры произвольной формы, а также шум (выбросы).

Сперва подберем гиперпараметры с помощью k-расстояние графа. Ищем локоть, в нем и будет оптимальный эpsilon (Листинг 13).

Листинг 13 — Подбор гиперпараметров

```
from sklearn.neighbors import NearestNeighbors
min_samples = 5
k = min_samples - 1
neigh = NearestNeighbors(n_neighbors=k)
nbrs = neigh.fit(X_scaled)
distances, indices = nbrs.kneighbors(X_scaled)
distances_to_kth = np.sort(distances[:, -1], axis=0)[:, -1]
plt.figure(figsize=(10, 5))
plt.plot(distances_to_kth)
plt.title('k-расстояния (k = 4)')
plt.xlabel('Объекты (отсортированы)')
plt.ylabel(f'Расстояние до {k}-го соседа')
plt.grid(True, alpha=0.5)
plt.show()
```

График изображен на Рисунке 8.

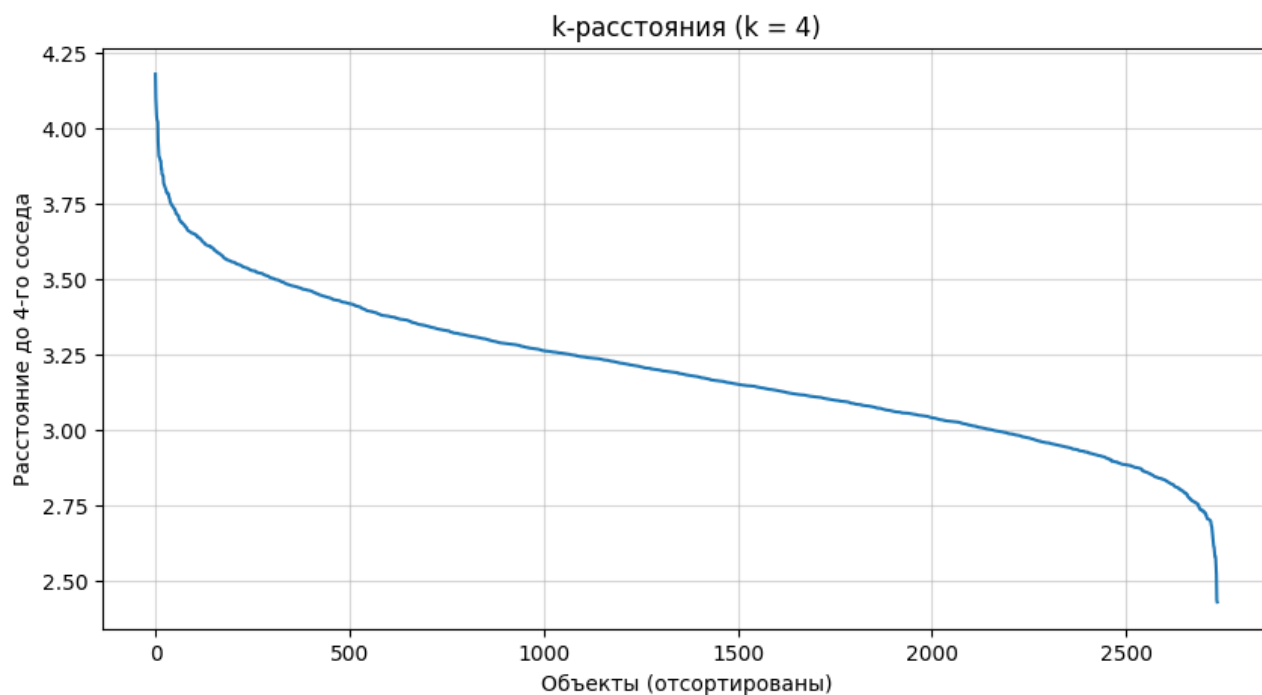


Рисунок 8 — Граф k-расстояния

Судя по локтю, оптимальным `eps` будет примерно 3.2. Проверим эту теорию (Листинг 14).

Листинг 14 — Поиск лучших гиперпараметров

```
def test_dbscan(X, eps, min_samples):
    start = time.time()
    db = DBSCAN(eps=eps, min_samples=min_samples)
    labels = db.fit_predict(X)
    time_taken = time.time() - start
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = list(labels).count(-1)
    if n_clusters >= 2:
        sil = silhouette_score(X, labels)
        db_idx = davies_bouldin_score(X, labels)
        ch_idx = calinski_harabasz_score(X, labels)
    else:
        sil = db_idx = ch_idx = np.nan
    return {
        'eps': eps,
        'min_samples': min_samples,
        'n_clusters': n_clusters,
        'n_noise': n_noise,
        'silhouette': sil,
        'davies_bouldin': db_idx,
        'calinski_harabasz': ch_idx,
        'time': time_taken,
        'labels': labels
    }

test_params = [
    (3.5, 5),
    (2.75, 10),
    (3.0, 5),
    (3.2, 5)
]

results = []
for eps, ms in test_params:
    res = test_dbscan(X_scaled, eps, ms)
    results.append(res)
    print(f"eps={eps}, min_samples={ms} -> кластеров: {res['n_clusters']}, шум: {res['n_noise']}, время: {res['time']:.2f} сек")
```

Получаем следующий вывод:

- `eps=3.5, min_samples=5` -> кластеров: 1, шум: 65, время: 0.03 сек;
- `eps=2.75, min_samples=10` -> кластеров: 0, шум: 2736, время: 0.01 сек;
- `eps=3.0, min_samples=5` -> кластеров: 37, шум: 1748, время: 0.01 сек;
- `eps=3.2, min_samples=5` -> кластеров: 3, шум: 620, время: 0.02 сек.

Убеждаемся, что ранее озвученный набор гиперпараметров действительно оказался самым оптимальным.

Посчитаем метрики для этого случая (Листинг 15).

Листинг 15 — Метрики для DBSCAN

```
valid_results = [r for r in results if r['n_clusters'] >= 2 and not
np.isnan(r['silhouette'])]
best = max(valid_results, key=lambda r: r['silhouette'])
print("Лучший результат DBSCAN:")
for k, v in best.items():
    if k != 'labels':
        print(f"{k}: {v}")
```

Нас интересуют следующие результаты:

- silhouette: -0.046170459525749497;
- davies_bouldin: 12.552899578823787 — выше, чем у иерархической кластеризации (1.64), что также говорит о худшем разделении;
- calinski_harabasz: 3.1527507822844005 — очень низкий, что подтверждает слабую межкластерную дисперсию.

Мы получили отрицательный коэффициент силуэта. Хотя он и близок к нулю, все равно это значит, что точки находятся ближе к другим кластерам, чем к своему.

Посмотрим визуализацию (Рисунок 9).

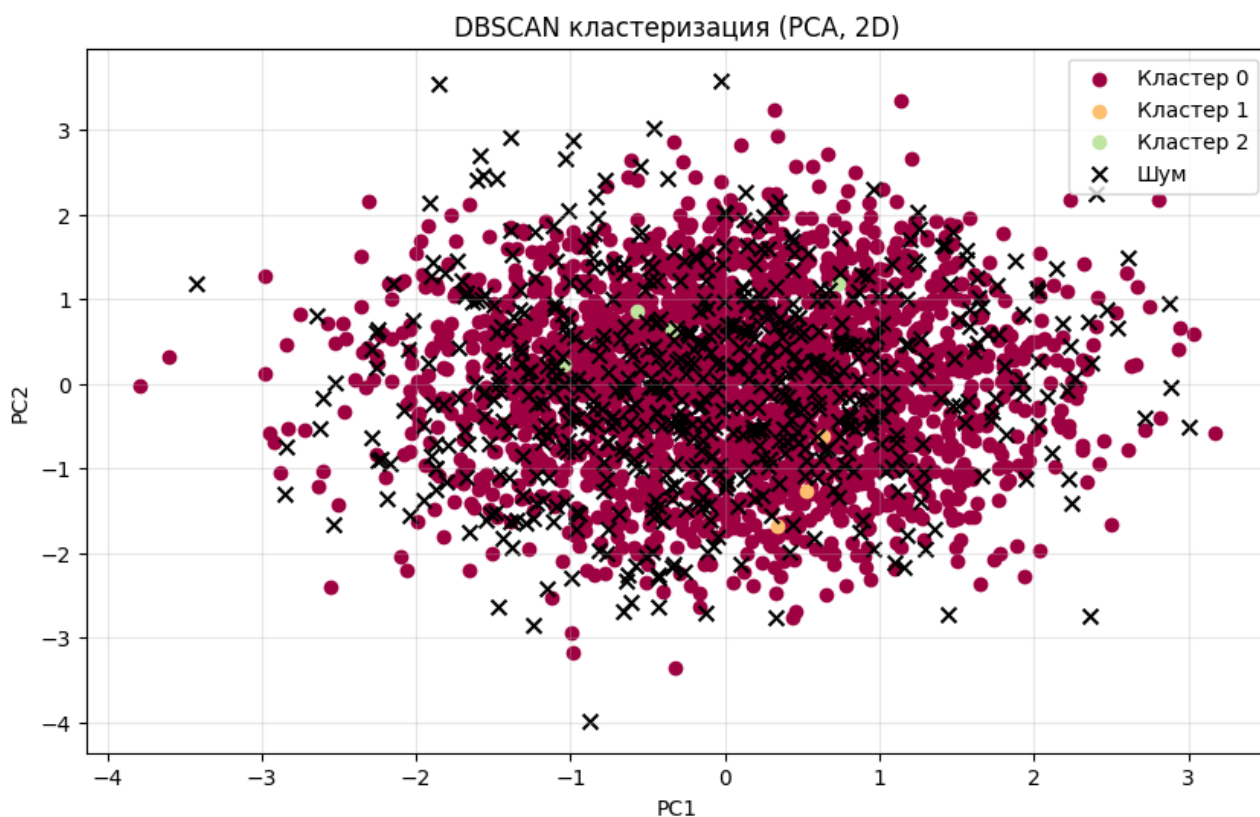


Рисунок 9 — Визуализация DBSCAN

На графике видим три кластера, которые тесно переплетены.

1.2 Задание 2. Снижение размерности

Применим PCA для снижения размерности до 2 и 3 компонент (Листинг 16).

Листинг 16 — Снижение размерности

```
from sklearn.decomposition import PCA
pca_2d = PCA(n_components=2)
pca_3d = PCA(n_components=3)
X_pca_2d = pca_2d.fit_transform(X_scaled)
X_pca_3d = pca_3d.fit_transform(X_scaled)
print("Объяснённая дисперсия (2D):", pca_2d.explained_variance_ratio_)
print("Суммарно (2D):", np.sum(pca_2d.explained_variance_ratio_))
print("\nОбъяснённая дисперсия (3D):", pca_3d.explained_variance_ratio_)
print("Суммарно (3D):", np.sum(pca_3d.explained_variance_ratio_))
```

Получаем следующий результат:

- объяснённая дисперсия (2D): [0.06612016 0.06510954];
- суммарно (2D): 0.13122970237686396;
- объяснённая дисперсия (3D): [0.06612016 0.06510954 0.06373867];
- суммарно (3D): 0.19496837194057706.

Это очень низкие значения. В идеале, если бы данные имели чёткую структуру (например, несколько кластеров), первые 2–3 компоненты объясняли бы >70–80% дисперсии. Здесь же — менее 20%.

Посмотрим на график накопленной дисперсии (Рисунок 10).

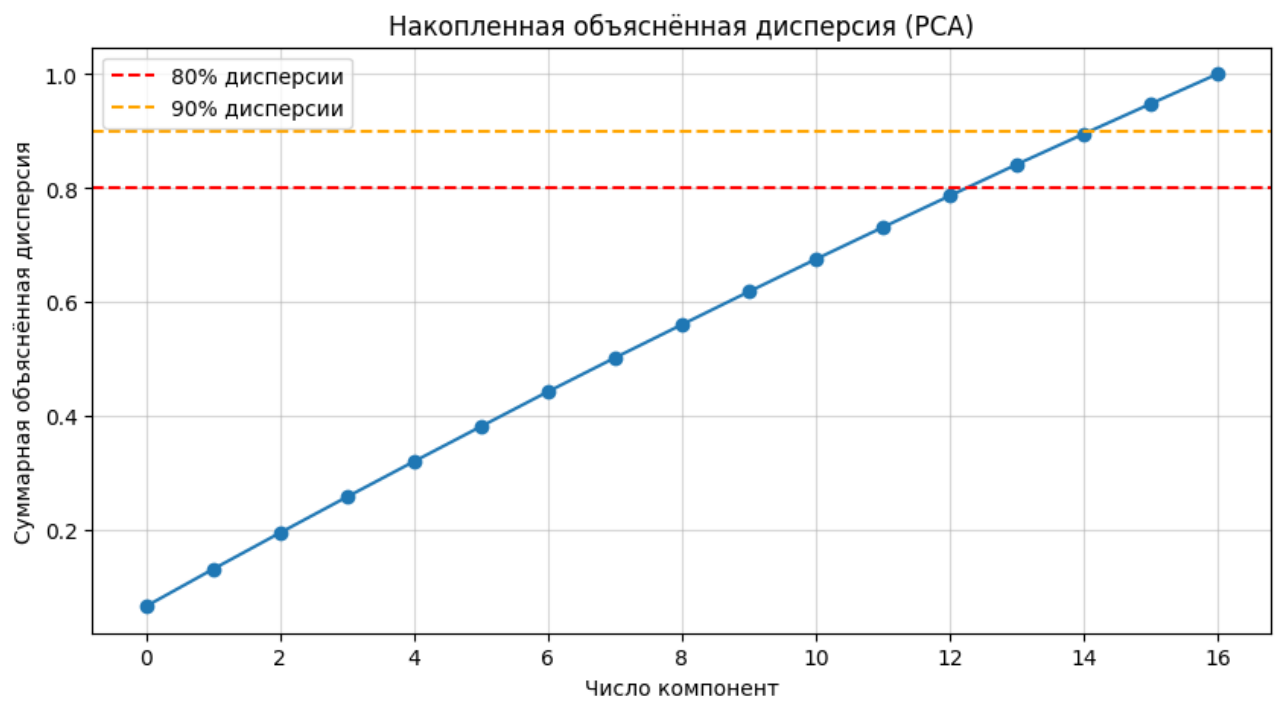


Рисунок 10 — График накопленной дисперсии

По нему мы видим, что нужно хотя бы 12 компонент, чтобы объяснить 80% дисперсии. Это говорит о том, что данные не имеют выраженной линейной структуры, признаки могут слабо коррелировать друг с другом.

Посмотрим на данные после PCA с двумя компонентами (Рисунок 11).

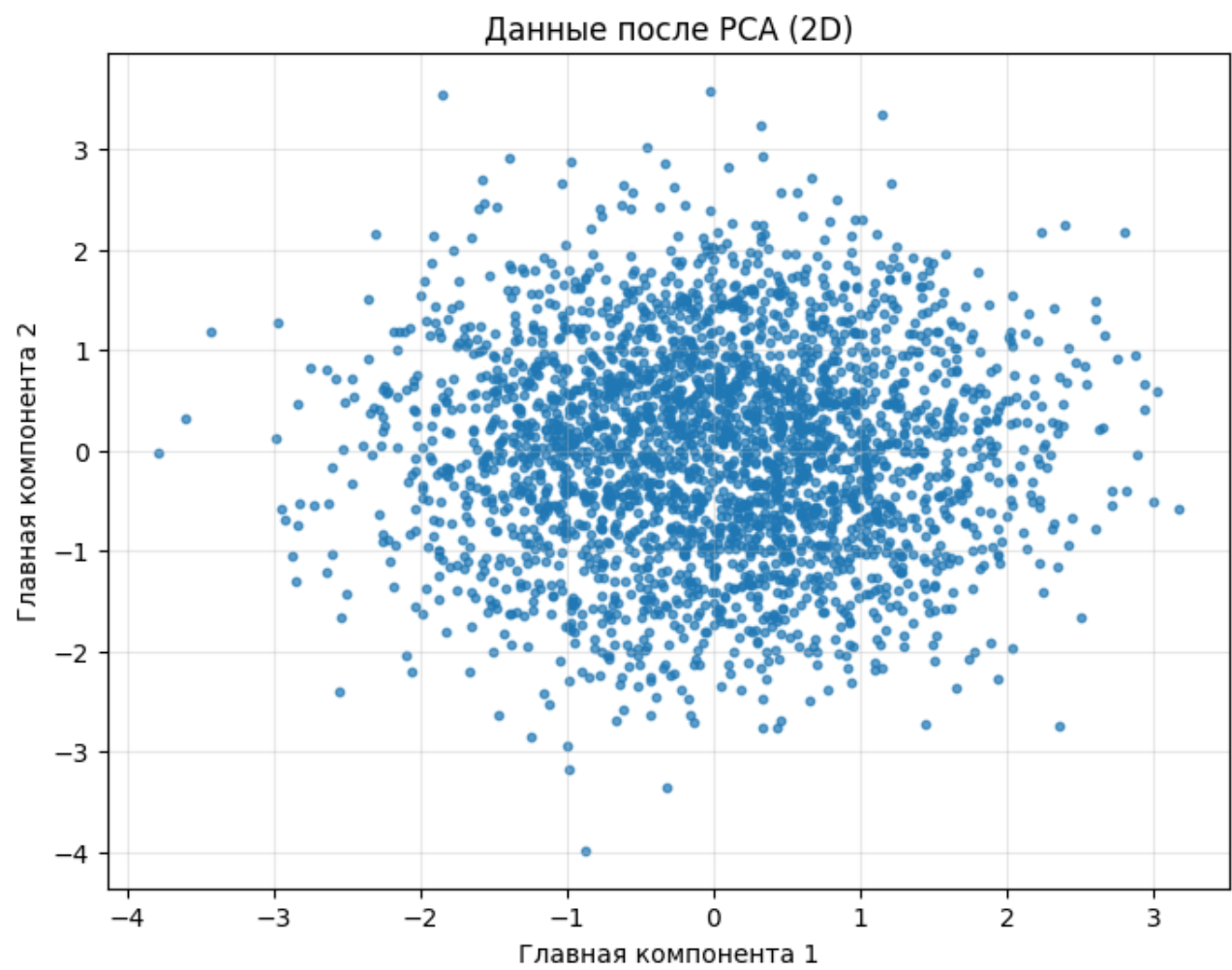


Рисунок 11 — Две компоненты

И с тремя компонентами (Рисунок 12).

Данные после PCA (3D)

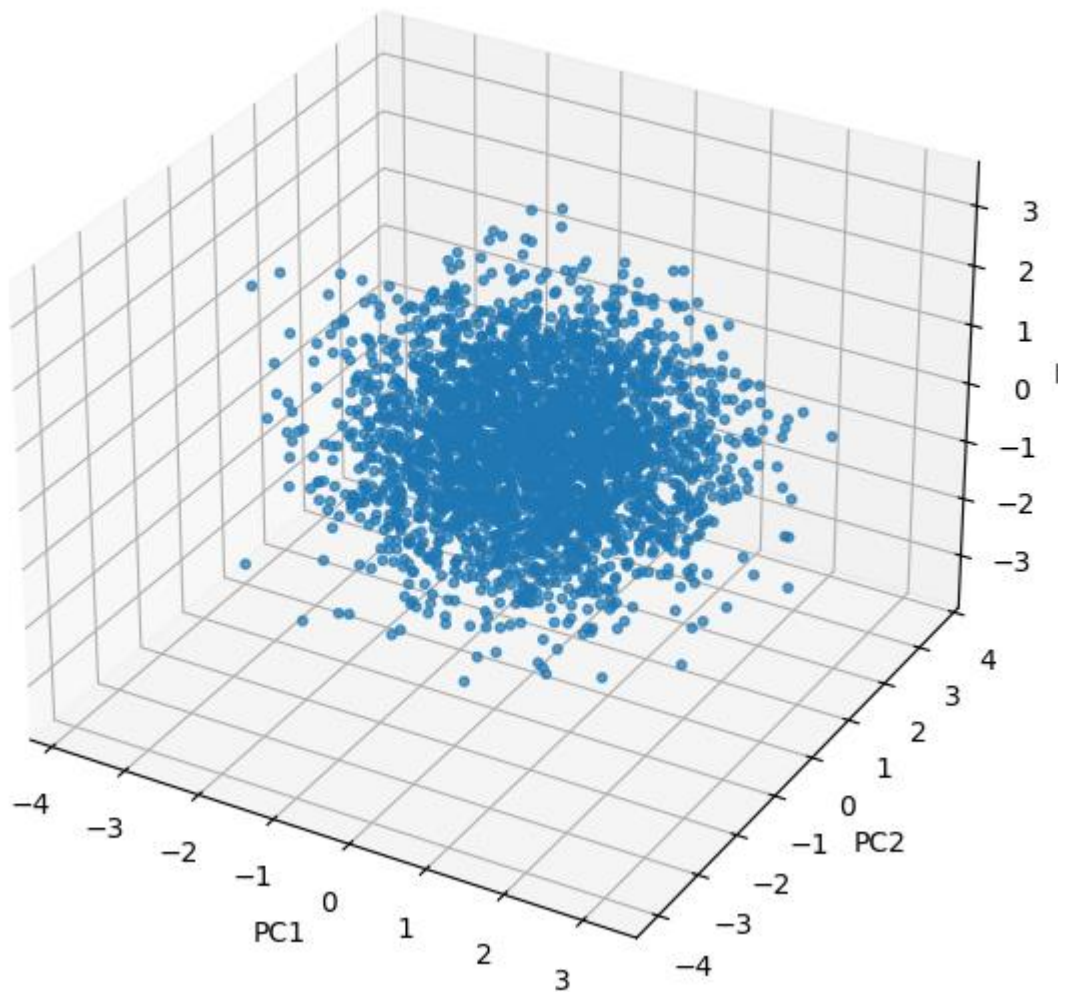


Рисунок 12 — Три компоненты

На графиках видно, что точки образуют одно плотное облако без явных подгрупп.

1.3 Задание 3. Влияние снижения размерности

Выполним KMeans на выборке после снижения размерности (Листинг 17).

Листинг 17 — Метод локтя и коэффициент силуэта

```
k_range = range(2, 11)
wcss = []
sil_scores = []
for k in k_range:
```

Продолжение Листинга 17

```
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_pca_2d_df)
wcss.append(kmeans.inertia_)
sil_scores.append(silhouette_score(X_pca_2d_df, labels))
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(k_range, wcss, marker='o')
plt.title('Метод локтя (PCA 2D)')
plt.xlabel('k')
plt.ylabel('WCSS')
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(k_range, sil_scores, marker='o', color='orange')
plt.title('Silhouette Score (PCA 2D)')
plt.xlabel('k')
plt.ylabel('Silhouette')
plt.grid()
plt.tight_layout()
plt.show()
best_k_pca = k_range[np.argmax(sil_scores)]
print(f"Лучшее k по силуэту на PCA(2D): {best_k_pca}")
```

Посмотрим полученные графики (Рисунок 13).

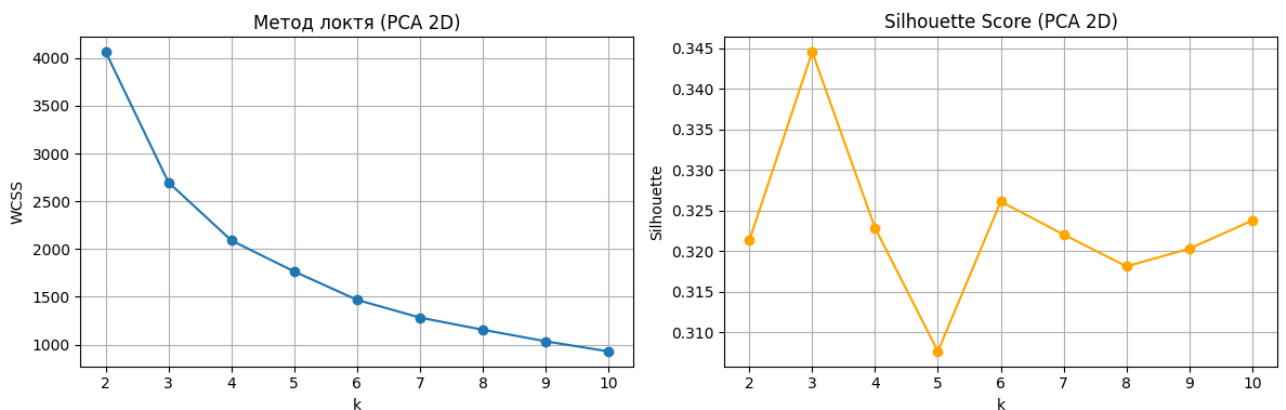


Рисунок 13 — Метод локтя и коэффициент силуэта

Уже видим прогресс, пиковое значение коэффициента достигло 0.345 при количестве кластеров, равном трем. Попробуем выявить метрики модели с тремя кластерами (Листинг 18).

Листинг 18 — Метрики KMeans после PCA

```
kmeans_pca = KMeans(n_clusters=best_k_pca, random_state=42, n_init=10)
start_time = time.time()
labels_kmeans_pca = kmeans_pca.fit_predict(X_pca_2d_df)
time_kmeans_pca = time.time() - start_time
sil_kmeans_pca = silhouette_score(X_pca_2d_df, labels_kmeans_pca)
db_kmeans_pca = davies_bouldin_score(X_pca_2d_df, labels_kmeans_pca)
ch_kmeans_pca = calinski_harabasz_score(X_pca_2d_df, labels_kmeans_pca)
print(f"K-means на PCA(2D) (k={best_k_pca}):")
print(f"Silhouette: {sil_kmeans_pca:.3f}")
print(f"Davies-Bouldin: {db_kmeans_pca:.3f}")
print(f"Calinski-Harabasz: {ch_kmeans_pca:.1f}")
```

```
print(f"Время: {time_kmeans_pca:.3f} сек")
```

Посчитанные метрики:

1. Davies-Bouldin: 0.920 — очень хорошее значение. Кластеры компактны и далеко друг от друга.
2. Calinski-Harabasz: 1725.7 — отличное значение. Сильная межкластерная дисперсия по сравнению с внутрикластерной.

Посмотрим визуализацию (Рисунок 14).

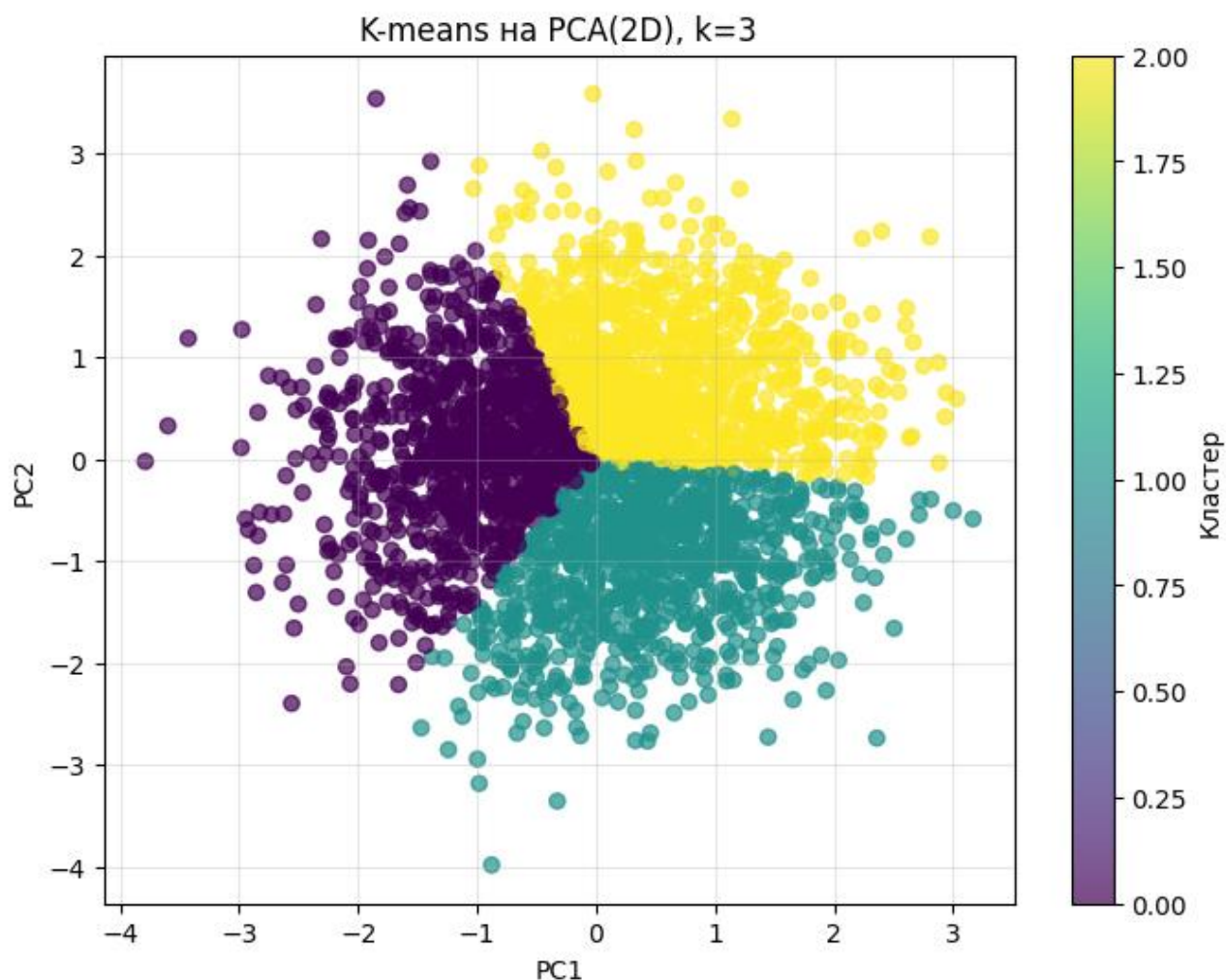


Рисунок 14 — Визуализация

Замечаем три ярковыраженных кластера, которые хорошо отделены друг от друга, имеются четкие границы. Это все визуально подтверждает высокие значения метрик.

ВЫВОД

В ходе выполнения практической работы №6 были успешно изучены алгоритмы кластеризации и методы снижения размерности, освоено их применение, а также разобрано, как снижение размерности влияет на работу кластеризационных алгоритмов.